

Presentation Notes & Graph Explanations

General Concepts

- g_c/T_M : Because for one block of B brought into registers, its generation cost is amortized across T_M rows of A .
- When T_N is larger, even if A leaves the cache, you need to reload it fewer times across the matrix, so the DRAM cold-fill cost is amortized.

Page 19: α Graph

- When T_M is small enough, the A tile fits in L1 with C completely. So there is no cost.
- When it starts spilling, you can calculate the penalty with $\left(\frac{\text{ram_lat} - \text{l1_lat}}{\text{reg}_m \cdot \text{reg}_k}\right) \times \frac{1}{T_N}$.
- From there, as long as C doesn't overflow the L1 cache, the size of T_M doesn't affect α .
- The second step is when C gets evicted, then you need to pay the read and write for it each time.

Page 20: α Graph

The Dip

- What is it? At first there is no amortization of the A because $T_M = 4$.
- As T_M grows a little we get amortization, so better performance.
- Then at $T_M = 16$ there is the jump because of the spillover.

Page 22: α Graph

- How do we calculate the T_M and T_N ? We calculate the α for T_M and T_N when g_c is 0 and create a table for each pair.
- For some $g_c \rightarrow$ we check each pair and calculate $\rightarrow \max\{\alpha \text{ (from table)}, g_c/T_M\} \rightarrow$ we pick the best pair out of the options.
- The α for large enough T_N as we've seen is quite low, and for a large T_M we can lower the g_c/T_M and almost always allow for a small α . That is possible because of the B residing in the FIFO.
- Most of the time the winner is mem-bound, not gen-bound. It becomes gen-bound when the g_c is too high.

Page 23: Ratios Graph

- T_M only goes up because we always want to lower the g_c/T_M as much as possible.
- T_N goes up at first because we want to amortize the cost of bringing in the evicted A tile we stream.
- T_N goes down when g_c is high enough because T_M is forced to grow to keep the cost down, and also we cannot allow T_N to be high enough for the C eviction, it's very bad, so lowering T_N is better even though it amortizes less. That makes sense because the T_M is large so we actually need less bands of A .

Page 25: Comparing to Square

Why lower T_N makes for bigger difference?

- Because the square just stays small, then there is almost no amortization for the huge g_c stall.

Why red goes down?

- Because at first it's already spilling out of C for low g_c . When g_c grows, because T_M is large for the square also, we get amortization and the difference is not so big for high g_c .

Why the green stays flat at some point?

- There they are both generation-bound; because of that, the ratio is just the ratio between the tile sizes: $1 - 8/12$.
- That stays until we get to a g_c where the optimal is not $T_M = 12$ anymore.

What happens with yellow?

- At first, the non-square can have a small enough T_M such that A just fits in the L1 cache with C , so it's faster.
- Then the non-square is gen-bound but the square is mem-bound, and the difference is not that big.
- After that, the square becomes gen-bound eventually and the difference starts growing.

What happens with blue?

- Same as yellow, just that the dip is longer, because T_N is larger, then T_M is larger (cause it's a square), meaning that it lowers the g_c/T_M for a longer time. Until the g_c is large enough such that the square loses to the new T_M of the optimal one.

Why the asymptote?

- Because at high g_c , T_M is big, but g_c is so big that g_c/T_M is still dominant and it becomes generator-bound again. Making it so the improvement is the ratio again:

$$\text{Speedup} \approx 1 - \frac{T_{M,\text{square}}}{T_{M,\text{optimal}}}$$

The place where the square and non-square are close, why is it that way?

- Because if you decrease T_M , the generator will stall you. If you increase T_M , you're at the point where it doesn't affect α anyway and you'll just cause C to spill. So staying near that square size is just the best option.